4. Operadores e Expressões

4.1 Operadores

"São elementos funcionais, que atuam sobre os operandos e produzem determinado resultado" (Saliba, 1993).

☞ Operadores Aritméticos

Atuam em expressões aritméticas, sobre operandos aritméticos e/ou constantes numéricas.

A tabela abaixo mostra: os **principais operadores aritméticos** disponíveis na **linguagem C++;** como eles são utilizados nas expressões aritméticas **(operação)**; sobre quantos operandos atuam **(tipo);** qual **sua prioridade de execução;** e o **resultado** obtido.

Operador	Operação	Tipo	Prioridade	Resultado
+	**Manter sinal**	Unário	1	**Positivo**
-	**Troca de sinal**	Unário	1	**Negativo**
*****	**Multiplicação** (inteira e ponto flutuante)	Binário	2	**Inteiro ou ponto flutuante**
/	**Divisão** (inteira e ponto flutuante)	Binário	2	**Inteiro ou ponto flutuante**
%	**Resto da Divisão** (somente inteiros)	Binário	2	**Inteiro**
+	**Soma** (inteira e ponto flutuante)	Binário	3	**Inteiro ou ponto flutuante**
-	**Subtração** (inteira e ponto flutuante)	Binário	3	**Inteiro ou ponto flutuante**

Observação:

Como pode ser observado na tabela acima, a linguagem C++ não possui um operador para **Exponenciação**. Exemplo: 2^3 cujo resultado é 8.

Isso ocorre porque a linguagem C também não tem, pois desde sua criação o C não foi pensado para executar operações matemáticas, principalmente, científicas.

Essa linguagem foi pensada e criada, por Dennis Ritchie e Ken Thompson para escrever um sistema operacional, e não para resolver problemas científicos. Além disso, era um dos objetivos da linguagem C melhorar o *Assembly* que, também, não possuía a exponenciação como instrução.

Além disso os computadores da década de 70 eram simples e, certamente, não compensava investir na criação de um operador para exponenciação, que não era uma operação comum, visto que na época os computadores eram usados para resolver problemas usando a aritmética simples.

Todavia, com o passar dos anos houve a necessidade de se fazer essa operação usando a linguagem C e, mais recentemente a C++. Portanto, para resolver esse problema foi criada uma

função matemática para exponenciação. Essa função é a **pow()**, e está **disponível na biblioteca math.h**, do compilador C++

A sintaxe dessa função é: **pow(base, potência);**

Atenção! Quando usado em expressões aritméticas, **essa função tem prioridade sobre** a multiplicação e divisão e, obviamente, sobre a adição e subtração.

☞ **Exemplo de um programa em C++, usando a função para exponenciação**

```
#include<stdio.h>
#include<stdlib.h>
#include<locale>
#include<math.h>

// Exponenciação

main()
{
   setlocale(LC_ALL,"Portuguese");

 // definição de variáveis

   int base, potencia, resultado;

   printf("\n<< Exponenciação de um Número em C++ >>");

   printf("\n\n Informe um Número.....: ");
   scanf("%d",&base);

   printf("\n\n Deseja elevar a qual potencia?.: ");
   scanf("%d",&potencia);

   resultado = pow(base,potencia);

   printf("\n\n Número %d elevado a potencia %d = %d",base, potencia, resultado);

   printf("\n\n");

   return 0;
}
```

A ordem de execução das operações depende da prioridade atribuída ao operador.

Exemplo:

Val_Total_Mercadoria := Val_Total_Mercadoria + Valor_Item * Aliquota_IPI;

A multiplicação será efetuada em primeiro lugar.

Importante!

A utilização de parênteses - () - modifica a prioridade do operador, ou seja, operações entre ( ) tem MAIOR prioridade.

Exemplo:

Total_NF = (Val_Total_Mercadoria - Desconto) * 1.10;

A subtração, embora tenha menor prioridade que a multiplicação, será executada por primeiro, pois está entre parênteses.

✓ **Exemplos para fixação:**

a) 20 + 10 + 15 + **24 / 6**
20 + 10 + 15 + 4 = **49**

b) 48 - **8 * 12** / 4 + 10
48 - **96 / 4** + 10
48 - **24 + 10**
48 - 34 = **14**

c) 10 * 10 % 3 - 1
100 % 3 - 1
1 – 1 = **0**

Operadores com prioridades idênticas, e que na expressão aritmética não aparecem entre parêntesis, então, a prioridade de resolução é a do primeiro operador que aparece na expressão.

✓ **Exemplos para fixação:**

9.0 * 4.5 + 3.5 / 2.5 * 2
40.5 + 3.5 / 2.5 * 2
40.5 + 1.4 * 2
40.5 + 2.8 = **43.3**

Operadores de Incremento e Decremento

A linguagem C++ permite a utilização dos operadores unários para incrementar (somar 1) ou decrementar (subtrair 1) em conteúdo de uma variável.

Operador	Operação	Sintaxe		Equivale a:
++	**Adiciona 1**	**Valor pré-fixado**	**++**VarX	VarX = VarX + 1;
		Valor pós-fixado	VarX**++**	
--	**Subtrai 1**	**Valor pré-fixado**	**--**VarX	VarX = VarX - 1;
		Valor pré-fixado	VarX**--**	

Conforme a sintaxe de declaração desses operadores, estes podem ser **pré-fixados** ou **pós-fixados**.

Quando são pré-fixados, eles **incrementam a variável antes de usar seu valor**.

Quando são pós-fixados, eles **utilizam o valor da variável antes de incrementá-la**.

Esses operadores possuem uma grande utilidade, principalmente na manipulação de variáveis contadoras, e em estruturas de repetição com variável de controle das repetições. Todavia, nem todas as linguagens de programação proporcionam essa funcionalidade.

☞ **Exemplo de um programa em C++, usando operadores de Incremento e Decremento**

```
#include<stdio.h>
#include<stdlib.h>

main()
{
  // Definição de variáveis

    int num1, resultado;

    printf("\n << Operadores de Incremento e Decremento >>");

    num1 =120;
    resultado = ++num1;                          // operador pré-fixado
    printf("\n\n Resultado ++var %d ",resultado);

    num1 = 120;
    resultado = num1++;                          // operador pós-fixado
    printf("\n\n Resultado var++ %d num1 = %d",resultado,num1);

    num1 = 120;
    resultado = --num1;                          // operador pré-fixado
    printf("\n\n Resultado --var %d ",resultado);

    num1 = 120;
    resultado = num1--;                          // operador pós-fixado

    printf("\n\n Resultado var-- = %d num1 = %d ",resultado,num1);

    printf("\n");

    return 0;
}
```

Resultado da execução do programa acima:

```
<< Operadores de Incremento e Decremento >>
Resultado ++var 121
Resultado var++ 120 num1 = 121
Resultado --var 119
Resultado var-- = 120 num1 = 119
--------------------------------
```

Analisando o resultado dessa execução podemos ver que no:

- ✓ 1º. resultado - **usando o operador de Incremento pré-fixado,** a variável num1 possuía um valor de 120, e antes de atribuir o seu conteúdo para a variável resultado, ela foi incrementada de 1, e a variável resultado recebeu então o conteúdo da num1 que é 121.
- ✓ 2º. resultado - **usando o operador de Incremento pós-fixado** a variável num1 possuía um valor de 120, e antes de incrementar 1 nela mesma, o seu conteúdo foi recebido pela variável resultado. Portanto, a variável resultado ficou com o valor inicial de num1 que era 120, e a variável num1 sofreu incremento de 1 ficando com valor de 121.
- ✓ 3º. resultado - **usando o operador de Decremento pré-fixado** a variável num1 possuía um valor de 120, e antes de atribuir o seu conteúdo para a variável resultado, ela (num1) foi decrementada (subtraída) do valor 1 e, então, resultado recebeu o valor de 121.
- ✓ 4º. resultado - **usando o operador de Decremento pós-fixado** a variável num1 possuía um valor de 120, e antes de decrementar 1 dela mesma, o seu conteúdo foi recebido pela variável resultado. Portanto, a variável resultado ficou com o valor inicial de num1 que era 120, e a variável num1 sofreu decremento de 1 ficando com valor de 119.

Expressões Reduzidas

São aplicadas nos casos que uma mesma variável se encontra em ambos os lados do comando de atribuição (**=**).

Expressões Reduzidas	Expressões Normais
VarX **+= 8**	VarX = VarX + 8;
VarX **-= 2.5**	VarX = VarX - 2.5;
VarX ***= 4**	VarX = VarX * 4;
VarX **/=** 7	VarX = VarX / 7;
VarX **%=** 8	VarX = VarX % 8;

Este tipo de expressão é muito útil para a manipulação de variáveis contadoras e/ou acumuladoras (que envolve somatórias, totais, etc.).

Exemplo de um programa em C++, usando Expressões Reduzidas

```
#include<stdio.h>
#include<stdlib.h>

main()

{
   // Definição de variáveis

   int num1 = 100;
   float valor = 87.5;

   printf("\n<< Teste Expressões Reduzidas >>");
```

```
    num1 += 8;
    printf("\n\n num1 = %d ",num1);

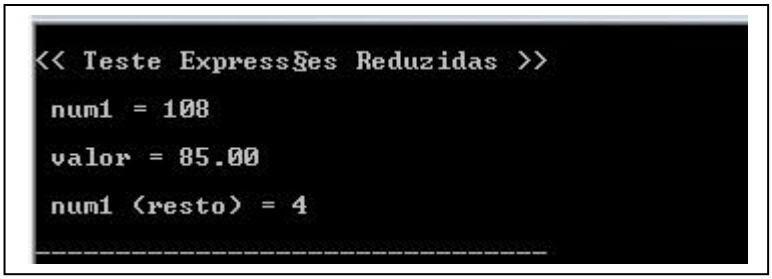
    valor -=2.5;
    printf("\n\n valor = %.2f ",valor);

    num1 %= 8;
    printf("\n\n num1 (resto) = %d ",num1);

    printf("\n");

    return 0;
}
```

Resultado da execução do programa acima:



Analisando o resultado dessa execução podemos ver que no:

Operadores Relacionais

Permitem comparar pares de operandos de **tipos de dados iguais**, resultando um valor lógico representado por um **bit 0** ou **1**. (**1** = Verdadeiro - **2** = Falso).

As comparações, normalmente, só podem ser feitas, variável com variável, ou variável com constante, sempre do mesmo Tipo de Dado.

Todavia, um Operando também pode ser uma Expressão Aritmética e, portanto, também pode ser comparado com variável ou constante.

Os operadores relacionais **comparam cada Byte contido em variáveis e/ou constantes**, e **retornam uma terceira variável de um Byte** (**variável booleana**), **como resultado** (Verdadeiro ou Falso).

Operadores Relacionais reconhecidos pelo compilador da linguagem C++

Operador	*Finalidade*
==	Igual
>	Maior
<	Menor
>=	Maior e igual
<=	Menor e igual
!=	Diferente

Atenção! - Operadores relacionais possuem a mesma prioridade. Assim sendo, não há necessidade de se preocupar em alterar sua prioridade em uma determinada condição.

☞ **Exemplo de um programa em C++, usando Expressões Relacionais**

```
#include<stdio.h>
#include<stdlib.h>

// Usando operadores relacionais

main()
{
   // definição de variáveis

    char nome1[6] = "PAuLo", nome2[6] = "PAULO";

    bool resultado;              // Variável Booleana armazena 1 = verdade ou 0 = falso

    char verdade[10] = "Verdade", falso[10] = "Falso";     // Cadeia de caracteres

    printf("\n<< Testes usando Operadores Relacionais - Dev-C++ >>");

    resultado = nome1 == nome2;    // resultado irá conter 0 ou 1

    /*
      Estrutura de decisão para mostrar a palavra Verdadeiro ou Falso, conforme resultado
      da expressão relacional anterior
   */

    if (resultado == 1)
       printf("\n\n %s = %s - %s",nome1, nome2,verdade);
    else
       printf("\n\n %s = %s - %s",nome1, nome2,falso);

    printf("\n");

    return 0;
}
```

☞ **Resultado da execução do programa acima:**

```
<< Testes usando Operadores Relacionais - Dev-C++ >>

 PAuLo = PAULO - Falso

--------------------------------
```

☞ **Exemplo de um programa em C++, usando Expressões Relacionais**

```
#include<stdio.h>
#include<stdlib.h>

// Usando operadores relacionais

main()
{
   // definição de variáveis

   int num1, num2;
   bool resultado;                          // 1 = verdade 0 = falso

   char verdade[10] = "Verdade", falso[10] = "Falso";

   printf("\n<< Testes usando Operadores Relacionais - Dev-C++ >>");

   num1 = 358;
   num2 = 75;

   resultado = num1 == num2;

   if (resultado == 1)
      printf("\n\n Num1 = %d e Num2 = %d - Num1 = Num2 - %s",num1, num2, verdade);
   else
      printf("\n\n Num1 = %d e Num2 = %d - Num1 = Num2 - %s",num1, num2, falso);

   printf("\n\n");

   resultado = num1 >= num2;

   if (resultado == 1)
      printf("\n\n Num1 = %d e Num2 = %d - Num1 >= Num2 - %s",num1, num2, verdade);
   else
      printf("\n\n Num1 = %d e Num2 = %d - Num1 >= Num2 - %s",num1, num2, falso);

   printf("\n");

   return 0;
}
```

Resultado da execução do programa acima:

```
<< Testes usando Operadores Relacionais - Dev-C++ >>
 Num1 = 358 e Num2 = 75 - Num1 = Num2 - Falso


 Num1 = 358 e Num2 = 75 - Num1 >= Num2 - Verdade
--------------------------------
```

☞ Operadores LÓGICOS

Na construção de algoritmos e consequentemente na programação de computadores existem ocasiões em que é necessário trabalhar com o relacionamento de duas ou mais condições (expressões relacionais) ao mesmo tempo, numa mesma instrução, realizando testes múltiplos. Por exemplo:

Se (salario_atual <= 1000.00) e (tempo_empresa > 5) então calcule reajuste;

Para esses casos é **necessária a utilização de operadores lógicos** ou booleanos.

☞ **Operadores lógicos atuam sobre resultados de expressões relacionais.**

A tabela abaixo mostra três operadores lógicos básicos, usados na linguagem C++.

Operador	Descrição	Tipo	Operação	Prioridade
!	NÃO	"Unário"	Negação	1
&&	E	Binário	Conjunção	2
||	OU	Binário	Disjunção	3

Importante! observar a prioridade dos operadores lógicos, dentro de uma expressão lógica.

Esses operadores são utilizados para a formação de novas proposições lógicas, compostas a partir de outras proposições lógicas simples.

☞ **Exemplo de um programa em C++, usando Expressões Lógicas**

```
#include<stdio.h>
#include<stdlib.h>
#include<locale.h>

// Usando operadores relacionais

main()

{
        setlocale(LC_ALL,"Portuguese");              // Mostra strings com acentos

   // definição de variáveis

   char letra1, letra2;
   bool resultado;                                   // 1 = verdade   0 = falso
   char verdade[10] = "Verdade", falso[10] = "Falso";

   letra1 = 'S';
   letra2 = 's';

   printf("\n<< Testes usando Operadores Lógicos - Dev-C++ >>");

   printf("\n\n letra1 = %c - letra2 = %c",letra1, letra2);
   printf("\n\n letra1 = letra2  E  letra2 = 's'");
```

```
   resultado = (letra1 == letra2) && (letra2 == 's');

    if (resultado == 1)
        printf("\n\n (%c = %c) && (%c = s) - %s",letra1, letra2, letra2, verdade);
    else
        printf("\n\n (%c = %c) && (%c = s) - %s",letra1, letra2, letra2, falso);

    printf("\n\n");
    system("pause");                    // Interrompe temporariamente a execução do programa

    printf("\n\n letra1 = %c - letra2 = %c",letra1, letra2);
    printf("\n\n letra1 != letra2  E  letra2 = 's'");

    resultado = (letra1 != letra2) && (letra2 == 's');

    if (resultado == 1)
        printf("\n\n (%c != %c) && (%c = s) - %s",letra1, letra2, letra2, verdade);
    else
        printf("\n\n (%c != %c) && (%c = s) - %s",letra1, letra2, letra2, falso);

    printf("\n\n");
    system("pause");

    system("cls");                      // limpa a tela
    printf("\n<< Testes usando Operadores Lógicos - Dev-C++ >>");

    printf("\n\n letra1 = %c - letra2 = %c",letra1, letra2);
    printf("\n\n letra1 != letra2  OU  letra2 = 's'");

    resultado = (letra1 != 'S') || (letra2 == 's');

    if (resultado == 1)
       printf("\n\n (%c != S) || (%c = s) - %s",letra1, letra2, verdade);
    else
       printf("\n\n (%c != S) || (%c = s) - %s",letra1, letra2, falso);

    printf("\n\n");
    system("pause");

    printf("\n\n letra1 = %c - letra2 = %c",letra1, letra2);
    printf("\n\n letra1 != letra2  OU  letra2 != 's'");

    resultado = (letra1 != 'S') || (letra2 != 's');

    if (resultado == 1)
       printf("\n\n (%c != S) || (%c != s) - %s",letra1, letra2, verdade);
    else
       printf("\n\n (%c != S) || (%c != s) - %s",letra1, letra2, falso);

    printf("\n");

    return 0;
}
```

Resultado da execução do programa acima:

```
<< Testes usando Operadores Lógicos - Dev-C++ >>
 letra1 = S - letra2 = s
 letra1 = letra2  E  letra2 = 's'
 (S = s) && (s = s) - Falso
Pressione qualquer tecla para continuar. . .

 letra1 = S - letra2 = s
 letra1 != letra2  E  letra2 = 's'
 (S != s) && (s = s) - Verdade
Pressione qualquer tecla para continuar. . .
```

```
<< Testes usando Operadores Lógicos - Dev-C++ >>
 letra1 = S - letra2 = s
 letra1 != letra2  OU  letra2 = 's'
 (S != S) || (s = s) - Verdade
Pressione qualquer tecla para continuar. . .

 letra1 = S - letra2 = s
 letra1 != letra2  OU  letra2 != 's'
 (S != S) || (s != s) - Falso
--------------------------------
```

Operador lógico **!** (não)

Esse operador nega um valor Booleano, ou seja, o uso do operador **NOT inverte o valor de seu operando**.

Exemplo: usando resultados **lógicos *ou* booleanos (0** ou **1)**

! (0) = **1**

! (1) = **0**

!(20 > 5) = **0**

!(20 < 5) = **1**

✓ Operador lógico && (E)

Também conhecido como **operador lógico de conjunção** (**E**) é utilizado quando dois ou mais relacionamentos lógicos de uma determinada condição necessitam ser verdadeiros para obter um resultado lógico verdadeiro; caso contrário, o valor do resultado lógico retornado será falso.

Portanto, em uma operação lógica **para que o resultado seja verdadeiro**, usando o operador lógico **&&**, é **necessário que os dois operandos tenham valor** lógico = **1** (verdadeiro).

Exemplo:

Supondo o conteúdo das variáveis:

Quantidade = 100

Preco = 20.00

E a expressão lógica:

(**Quantidade >** 80) **&&** (**Preco =** 50,00).

Uma vez executada, essa expressão lógica Irá apresentar resultado **0** (falso), pois somente o operando 1 retorna um resultado = **1** (verdadeiro).

Se o conteúdo da variável Preço fosse igual a 50.00, então o resultado da expressão lógica seria **1** (verdadeiro).

✓ Operador lógico || (OU)

Também conhecido como **operador lógico de disjunção** (**OU**) é utilizado quando pelo menos um dos relacionamentos lógicos de uma condição necessita ser verdadeiros para obter-se um resultado lógico verdadeiro; caso contrário, o valor do resultado lógico retornado será falso.

Portanto, em uma expressão lógica **para que o resultado seja verdadeiro**, usando o operador lógico **||**, **basta que um dos dois operandos retornem um valor** lógico = **1** (verdadeiro).

Exemplo:

Supondo o conteúdo das variáveis:

Quantidade = 100 e

Preço = 20.00.

E a expressão lógica:

(Quantidade > 80) **||** (**Preço =** 50.00)

Uma vez executada, essa expressão lógica Irá apresentar resultado = **1** (verdadeiro), pois embora o operando 2 retorne um valor = **0** (falso), o operando 1 retornará um valor = **1** (verdadeiro).

☞ Tabelas-verdade (Operadores Lógicos)

As tabelas-verdade, montadas abaixo, têm por objetivo dispor de uma forma prática os valores lógicos envolvidos em uma expressão lógica.

✓ Operação de Negação – !

Valor do operador A	Resultado da Operação **!**(A)
0	**1**
1	**0**

✓ **Operação de Conjunção – &&** (E)

Valor do operador		Resultado da Operação (A) **&&** (B)
A	B	
0	**0**	**0**
0	**1**	**0**
1	**0**	**0**
1	**1**	**1**

✓ **Operação de disjunção** (não-exclusiva) **– ||** (OU)

Valor do operador		Resultado da Operação (A) **||** (B)
A	B	
0	**0**	**0**
0	**1**	**1**
1	**0**	**1**
1	**1**	**1**

☞ **Prioridade entre Operadores**

	Prioridade
Parênteses mais internos	**1**
Operadores aritméticos	**2**
Operadores relacionais	**3**
Operadores Lógicos	**4**

Exemplo:

Qual seria o resultado da expressão abaixo: 1 ou 0 ?

!((5 <> 10 / 2 **) || 1 &&** 2 – 5 > 5 – 2 **|| 1)**

Solução:

!(5 <> 5) **|| 1 &&** -3 > 3 **|| 1)**

!(0 || 1 && 0 || 1)

!(0 || 0 || 1)

!(0 || 1)

!(1) = 0

4.2 Expressões

Uma expressão pode ser definida como a combinação de variáveis, constantes e operadores, que processadas, produzem um valor.

Exemplo:

Salario_Mensal = Horas_Trabalhadas ***** Salario_Hora **+ 5.00;**

(Salario_Mensal > Salario_Base)

(Salario_Mensal >= **1000.00**) && (Salario_Mensal <= **5000.00**)

☞ Expressões Aritméticas

Expressões aritméticas são aquelas cujos **operadores são aritméticos** e cujos **operandos** são **constantes ou variáveis do tipo numérico** (tipo de dado inteiro ou com decimais).

Exemplo:

Valor_Produto **= (**Qtd_Produto ***** Preço_Produto **)** * 0.9**;**

Media_Final **= (**Nota1 + Nota2 + Nota3**)** / 3;

☞ Expressões Relacionais

Expressões relacionais são aquelas cujos **operadores são relacionais** e cujos **operandos** podem ser expressões aritméticas, variáveis e/ou constantes.

Expressões relacionais produzem resultado lógico (booleano) Verdadeiro (**1**) ou Falso (**0**).

Exemplos**: Supondo** Var_A = 20 Var_B = -5 VarC = 15

a. **Var_A > Var_B** - Resultado = **1**

b. **(Var_A + Var_B * Var_C) = 100** - Resultado = **0**

c. **“Pedro” <> “João”** - Resultado = **1**

☞ Expressões Lógicas

Expressões lógicas são aquelas cujos operadores são: lógicos de conjunção ou disjunção, **e cujos** operandos são expressões relacionais, variáveis e/ou constantes.

Temos também o operador de negação, usado para inverter o resultado de um expressão lógica.

Expressões lógicas produzem como resultado um valor lógico: 1 (Verdadeiro) **ou 0** (Falso)**.**

Exemplos: Supondo **Var_A = 20 Var_B = -5 VarC = 15**

a. **!(Var_A >= Var_B)** - Resultado = **0**

b. **(Var_A > Var_B) && (Var_B < Var_C)** - Resultado = **1**

c. **(Var_A < Var_B) || (Var_B < Var_C)** - Resultado = **1**

d. **(Var_A < Var_B) || (Var_B > Var_C)** - Resultado = **0**

Exercício de fixação:

Elaborar algoritmos na forma de um programa na linguagem C++, para:

1. Calcular um reajuste de 12.28% sobre o Salário de um funcionário.
   Entrada: Salário Atual do Funcionário.
   Saída: Salário Atual (anterior ao reajuste); Valor do Reajuste; e o Salário Reajustado.

2. Calcular o Valor do ICMS de uma Nota Fiscal.
   Entrada: Valor Total da Nota, Alíquota do ICMS.
   Saída: Valor Total da Nota; Alíquota do ICMS; e o Valor do ICMS.

3. Calcular e mostrar a área de um retângulo.
   Entrada: Comprimento e Largura do retângulo.
   Saída: Área do retângulo.

4. Calcular a área de um terreno retangular e o total da área construída, supondo que esta ocupa 70% da área total do terreno.
   Saída: Área Total do Terreno; área construída; e área livre (tudo em metros quadrados).